



BSc (Hons) in **Computer Science**  
**SE3062 : Intelligent Systems**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 02

**Objective & Lecture Mapping:** In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

### Part 1: Practical Implementation — Coding the Architectures

#### Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates ( `agent_pos` ). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

#### Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*

Example Logic: IF food\_here THEN suck; IF wall\_ahead THEN turn\_left; ELSE move\_forward

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

### Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.  
Example Logic: IF wall\_ahead AND left\_is\_visited THEN turn\_right
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

## Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent would need to store an action for every possible percept sequence. As the agent's lifetime increases, the number of possible percept sequences grows combinatorially. For complex environments such as Chess, the number of possible states and percept histories is enormous, making the table physically impossible to build and store. This is known as combinatorial explosion.

**2. (Understand) Look at the code you wrote for your `SimpleReflexAgent` . Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.**

```
The condition-action rules are the if/elif/else statements inside sense_and_act():
if percept.get("food_here"):
    return "suck"
elif percept.get("wall_ahead"):
    return "turn_left"
else:
    return "move_forward"
```

These directly map conditions from the current percept to actions, which is the defining behaviour of a Simple Reflex Agent.

**3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?**

The Simple Reflex Agent got stuck because the environment is Partially Observable and the agent has no percept history or internal memory.

When it encounters the same situation, it receives the same percept and therefore chooses the same action. In a boxed-in corner, every direction is blocked, so the rule:

IF wall\_ahead THEN turn\_left

causes the agent to repeatedly turn left forever.

The agent cannot remember that it has already tried the same situation.

**4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent` . Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).**

The Sensor Model uses the incoming percepts such as wall\_ahead and food\_here to update the agent's internal state.

The Transition Model represents how actions are expected to change the agent's situation. For example, move\_forward should change the agent's position, while turning changes its heading without changing its position.

The Model-Based Agent uses this internal state, including last\_action and consecutive\_turns, before selecting its next action. Therefore, its decision depends on

Finalize and Lock Lab 02 Documentation

**Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.**



**FACULTY OF COMPUTING**